

ATOMOD

Guide d'utilisation complet

Application d'.....

H. Bulou

Version 0.6.0
2026-03-13

Table des matières

1	Sauvegarde des données générées	5
2	Le modèle EXAFS	7
3	Le modèle ATOMOD	9
3.1	Le modèle <code>build_atomod_v2</code>	9
3.2	La méthode <code>model.fit</code>	10
3.2.1	Fonction de la méthode <code>model.fit</code>	10
3.2.2	Arguments principaux de la méthode <code>model.fit</code>	11
3.3	Le <code>"train_generator"</code>	12
3.4	Le <code>"validation_generator"</code>	12
3.5	L'encodeur EXAFS	13
3.6	Le UNet de reconstruction	14
4	Utilisation du code FEFF dans le projet M2P2_HEA	15
4.1	Objectif scientifique	15
4.2	Méthodologie de Simulation	15
4.3	Structure du fichier de données générées par FEFF	18
A	Glossaire	19

Chapitre 1

Sauvegarde des données générées

ATOMOD génère un grand nombre de données différentes :

- des nanoparticules NP. Chacune d'elle est caractérisée par une liste d'atomes la composant.
- chacun de ces atomes est caractérisé lui-même par des données telles que
 - sa nature chimique (chaîne de caractère),
 - sa position (vecteur de dimension 3),
 - son énergie de site,
 - son nombre de coordination **CN**, c'est à dire simplement le nombre de voisins directs de l'atome, sans aucune information sur la nature de ses voisins,
 - son nombre de coordination généralisé, **GNN**, qui pondère le **CN** par le **CN** de chacun de ses voisins :

$$\text{GCN}_i = \frac{1}{\text{CN}_{\max}} \sum_{j \text{ voisin de } i} \text{CN}_j$$

- la composition chimique de son environnement local,
- son spectre EXAFS.

Chapitre 2

Le modèle EXAFS

Le modèle **EXAFS** génère des spectres EXAFS et l'énergie de site d'atomes à partir d'informations tels que leur nature chimique, leur nombre de coordination et leur environnement local c'est à dire la fraction de chaque espèce dans la sphère de coordination.

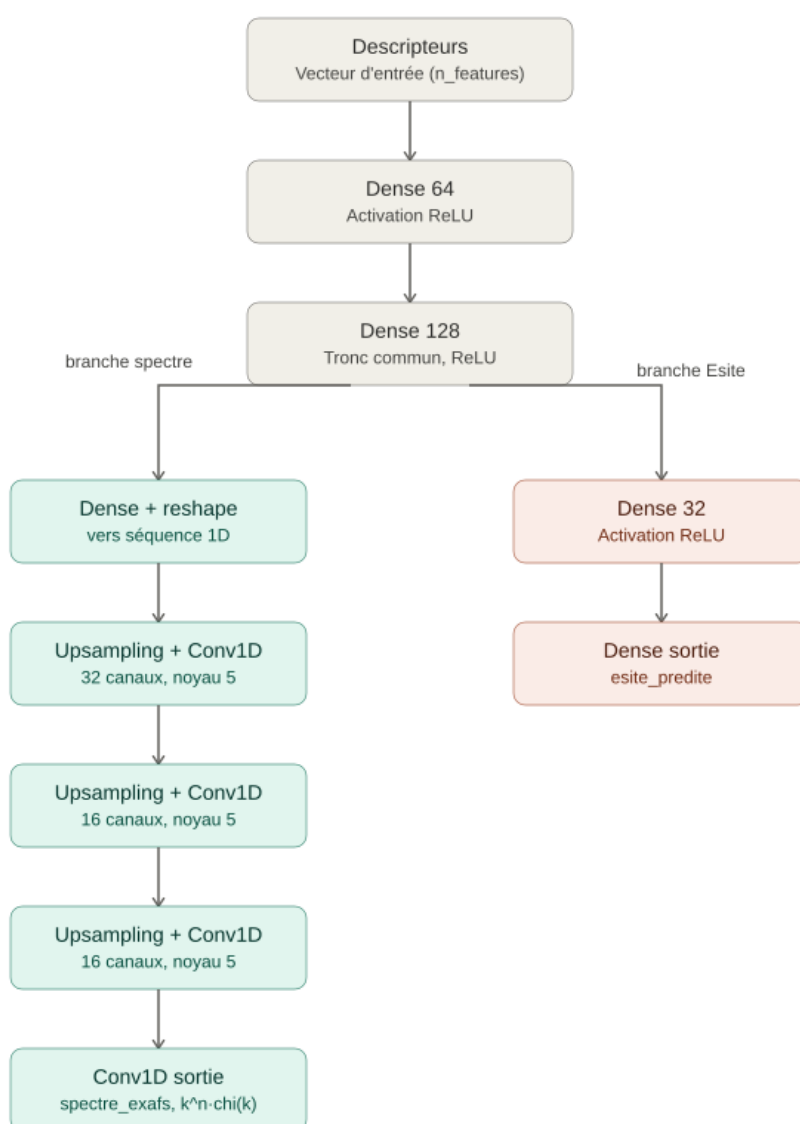


FIG. 2.1

Il s'agit d'une architecture encodeur dense $\rightarrow$ décodeur convolutif 1D.

Chapitre 3

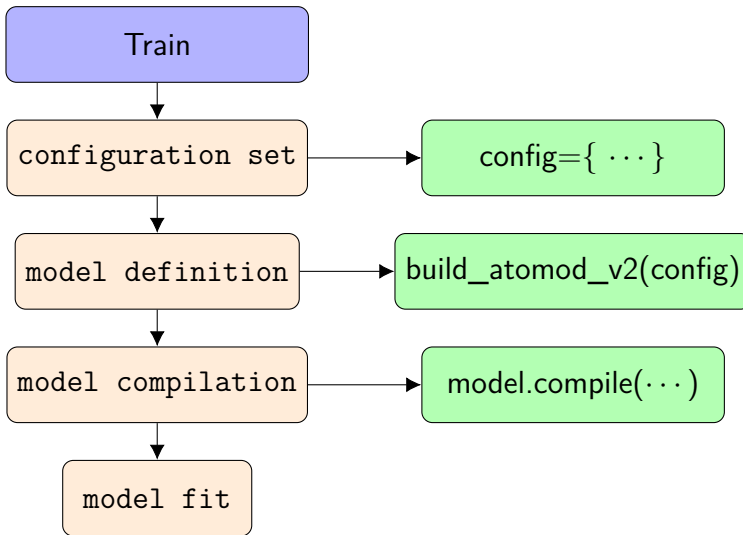
Le modèle ATOMOD

La finalité de l'application ATOMOD est de reconstruire une structure 3D à partir d'informations projetées 2D (TEM) et intégrées statistiquement (EXAFS global) à une carte atomique 3D discrète d'un alliage à haute entropie (HEA), où le désordre chimique est maximal.

Dans ce contexte, nous avons développé une approche basée sur une architecture de type UNet, modifiée avec du conditionnement "FiLM" (Feature-wise Linear Modulation). C'est une approche basée sur une architecture de type UNet 2D conditionnée par un Encodeur EXAFS. Elle permet que l'information spectroscopique guide la reconstruction d'image tout au long du processus. L'image TEM donne les positions (x, y) , l'EXAFS donne les distances locales pour la dimension z et la chimie. Le principe de fonctionnement est qu'à chaque étage de UNet, on injecte l'information EXAFS, qui va influencer ou "moduler" la manière dont l'image est traitée pendant le processus de calcul. Le spectre peut forcer le UNet à activer certains canaux de sortie (certains plans atomiques z) qu'il aurait ignorés autrement.

À chaque étage de l'encodeur et du décodeur du UNet, on utilise le vecteur latent EXAFS pour générer des paramètres γ et β qui vont normaliser et décaler les cartes de caractéristiques (feature maps) de l'image.

$$\text{FiLM}(F) = \gamma(\text{Latent_EXAFS}) \cdot F + \beta(\text{Latent_EXAFS}) \quad (3.1)$$



3.1 Le modèle build_atomod_v2

L'architecture du modèle est définie dans la fonction `build_atomod_v2()`.

```
model = build_atomod_v2(config)    # instantiation du modèle
```

L'appel `model = build_atomod_v2(config)` crée le modèle en mémoire, alloue les matrices pour les poids des neurones, et leur donne des valeurs aléatoires initiales. C'est l'instanciation du modèle, c'est à dire la création concrète de l'objet informatique dans la mémoire de l'ordinateur, à partir de son plan théorique (sa classe ou son architecture).

Ensuite, une fois le modèle instancié, on peut

- l'entraîner (`model.fit()`),
- voir son résumé (`model.summary()`),
- faire des prédictions (`model.predict()`).

Après avoir instancié le modèle, on le compile (méthode `model.compile()`) c'est à dire que l'on configure la stratégie que le réseau va utiliser pour apprendre. Alors que l'instanciation consiste à créer le corps du réseau de neurones en mémoire, la compilation consiste à lui donner son "cerveau" et ses règles de calcul avant qu'il ne commence l'entraînement. Lors de la compilation, on définit les trois piliers essentiels de l'apprentissage du réseau :

- L'Optimiseur (`optimizer`) c'est à dire comment vont se faire les ajustement du modèle lors de l'apprentissage. C'est l'algorithme mathématique qui va mettre à jour les poids des neurones en fonction des erreurs commises. L'optimiseur décide de combien et dans quelle direction modifier les curseurs du réseau pour qu'il devienne plus précis au lot suivant. Il existe plusieurs optimiseurs comme `adam` ou `sgd`.
- La Fonction de Perte (`loss`) qui définit la formule mathématique qui mesure l'écart entre ce que le modèle a prédit et la vérité terrain. Elle donne une note de mauvaise performance. Plus la `loss` est élevée, plus le modèle s'est trompé. Le but du réseau sera de minimiser cette valeur.
- Les Métriques (`metrics`) qui sont les indicateurs à suivre pendant l'entraînement pour juger si le modèle progresse. A la différence de la `loss` qui sert au calcul pur du GPU, les métriques rassurer ou alerter sur les performances réelles du modèle. Par exemple, `accuracy` indique le taux de bonne classification des pixels.

```
# Compilation du modèle.
# étape obligatoire AVANT de pouvoir lancer le model.fit()
model.compile(
    optimizer=config['optimizer'],    # Algorithme d'apprentissage
    loss=combined_loss,               # règle de physique/segmentation
    metrics=['accuracy']              # Ce que l'on veut surveiller
)
```

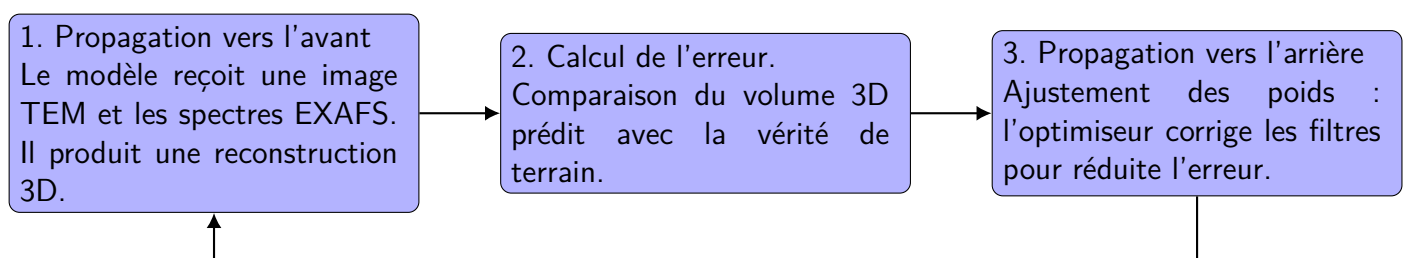
Une fois compilé, TensorFlow fige la structure mathématique du réseau et génère les graphiques de calcul optimisés (les gradients). L'étape suivante consiste à fournir au modèle les données nécessaires à l'entraînement (images TEM, spectres EXAFS, etc.).

3.2 La méthode `model.fit`

3.2.1 Fonction de la méthode `model.fit`

Dans l'écosystème de TensorFlow/Keras, l'instruction `model.fit()` est la commande qui lance l'entraînement (l'instruction) du réseau de neurones.

Avec l'exécution de `model.fit(train_generator, epochs=50)`, le réseau entre dans une boucle itérative qui reproduit le mécanisme de l'apprentissage humain par essai-erreur :



Ce cycle ($1 \rightarrow 2 \rightarrow 3$) est répété pour chaque lot (**batch**) de nanoparticules, et ce sur l'ensemble des données, autant de fois que le nombre d'époques spécifié (**epochs=50**).

La méthode `model.fit()` gère :

- La logistique des données car c'est `model.fit()` qui appelle la méthode `__getitem__` de `train_generator` à chaque étape pour charger les données, et qui s'assure qu'elles soient envoyées au processeur (CPU ou GPU) pour le calcul.
- Le suivi en temps réel : À chaque itération, `model.fit()` affiche la valeur actuelle de la loss et de l'accuracy.
- L'évaluation de validation : À la fin de chaque époque, il bascule automatiquement sur `val_generator` pour mesurer la capacité du modèle à généraliser sur de nouvelles nanoparticules non considérées pour l'instruction.
- Le pilotage des **Callbacks** : C'est lui qui orchestre les outils de contrôle (par exemple, appel au `ModelCheckpoint` pour sauvegarder le modèle si le score est bon, et envoie l'image au `ImageSamplingCallback` pour générer une image png de contrôle).

`history=model.fit(...)` renvoie un objet **history** contenant l'historique complet des valeurs de pertes et de métriques pour chaque époque. On peut l'utiliser juste après pour tracer des courbes et vérifier graphiquement si le modèle a bien convergé ou s'il a sur-appris.

3.2.2 Arguments principaux de la méthode `model.fit`

1. Les arguments de Données (Les entrées)

- **x** : les données d'entrée **input** utilisées pour l'entraînement. Il s'agit du générateur de données d'entraînement `train_generator` qui fournit le couple `[batch_tem, batch_exafs]`.
- **y** : Les données cibles **target**. Cet argument est laissé à `None` car le générateur `train_generator` fournit **x** et **y** en même temps sous la forme de tuple `( [batch_tem, batch_exafs], batch_volume_target )`.

Par exemple, à chaque itération, `train_generator` envoie

- **batch_tem** : Les 4 images TEM (une par nanoparticule) de forme `(4, 128, 128, 1)`,
- **batch_exafs** : Les 4 matrices de spectres normalisés (une par nanoparticule) de forme `(4, N_points, N_especes)`,
- **batch_volume_target** : Les 4 volumes 3D cibles (un par nanoparticule) de forme `(4, 128, 128, N_especes * N_plans)`.

2. Les arguments de Configuration du Cycle d'entraînement

- **batch_size** est le nombre d'échantillons traités avant chaque mise à jour des poids du réseau. Note : Si on passe un générateur à **x**, cet argument n'est pas utilisé ici car la taille des lots est directement définie à l'intérieur du générateur.
- **epochs** est le nombre total de fois où le modèle va parcourir l'intégralité du jeu de données d'entraînement.
- **steps_per_epoch** est nombre total de lots (itérations) à faire défiler avant de déclarer qu'une époque est terminée. Généralement, on lui donne la valeur `len(train_generator)`. Si le générateur contient 1000 lots, il fera 1000 étapes par époque.

3. Les arguments de validation

4. Les arguments de contrôle et d'affichage

3.3 Le "train_generator"

Dans la fonction `model.fit()`, `train_generator` représente le générateur de données qui sert à alimenter le réseau de neurones en cours d'entraînement. Il prépare, nettoie, normalise et livre les données par petits paquets directement utilisables par le calcul mathématique de `model.fit`. Plutôt que de charger toutes les images TEM, volumes 3D et spectres EXAFS en mémoire RAM d'un coup (ce qui risquerait de "planter" l'ordinateur en raison du volume massif de données de simulation), le générateur fonctionne selon le principe du chargement à la volée ("on-the-fly"). Durant l'entraînement, le `train_generator` découpe les données servant à l'entraînement en lots (`Batches`). Par exemple au lieu de donner les images TEM, les spectres EXAFS et les volume 3D de probabilité de présence atomique des 10 000 nanoparticules d'un coup, `train_generator` regroupe les données par petits paquets. si le `BATCH_SIZE` est fixé à 4, le `train_generator`

- lit 4 images TEM (une par nanoparticules),
- lit $4 \times N_ESPECES$ spectres EXAFS,
- lit $4 \times N_ESPECES$ volumes 3D des probabilités de présence atomique cibles,
- assemble toutes ces données dans des tableaux NumPy et le envoie au GPU.

A chaque itération, `train_gen` fournit un couple strict de données sous la forme (`[batch_tem, batch_exafs], batch_volume_target`) :

- `batch_tem` : Les 4 images TEM de forme (4, 128, 128, 1),
- `batch_exafs` : Les 4 matrices de spectres normalisés de forme (4, `N_points`, 3),
- `batch_volume_target` : Les 4 volumes 3D cibles de forme (4, 128, 128, `N_especes * N_plans`).

Grâce à la fonction `on_epoch_end()` intégrée dans les générateurs `Keras`, dès qu'une époque se termine, `train_generator` mélange l'ordre des nanoparticules. Ainsi, à l'époque suivante, le réseau ne verra pas les données dans le même ordre, ce qui améliore grandement la robustesse de l'apprentissage et évite le sur-apprentissage.

3.4 Le "validation_generator"

Dans la fonction `model.fit()`, `validation_generator` représente le générateur des données de validation.

Il fonctionne exactement sur le même principe logistique que `train_generator` (chargement et normalisation à la volée par lots), mais il sert un objectif complètement différent : évaluer les performances réelles du modèle sur des nanoparticules qu'il n'a jamais vues pendant son entraînement.

`validation_generator` sert à

- Détecter le Sur-apprentissage (Overfitting) : C'est le plus grand danger avec les données simulées *in silico*. Le modèle est très intelligent : il est capable d'apprendre par cœur les pixels exacts de des images d'entraînement. Si la perte de `train_generator` baisse mais que la perte de `validation_generator` stagne ou remonte, le modèle est en train d'apprendre par cœur. Il devient inutile pour de futures vraies images TEM.
- Piloter les Callbacks : C'est grâce aux résultats obtenus sur `validation_generator` que le callback `ModelCheckpoint("best_atomod.h5", save_best_only=True)` sait s'il doit sauvegarder le modèle. Il ne sauvegarde que si le score sur le jeu de validation s'améliore.
- Arrêter l'entraînement au bon moment : Le callback `EarlyStopping` surveille `validation_generator`. Si l'erreur de validation ne baisse plus pendant 10 époques d'affilée, il coupe l'entraînement automatiquement pour éviter de perdre du temps et d'endommager la généralisation du réseau.

Contrairement à `train_generator`, `validation_generator` ne doit jamais mélanger ses données de manière aléatoire en cours de route (`shuffle=False`). On veut que l'évaluation soit fixe et rigoureuse à chaque fin d'époque pour comparer équitablement les scores.

3.5 L'encodeur EXAFS

Le but de l'encodeur est de condenser le signal 1D (qui contient les fréquences d'oscillation liées aux distances interatomiques et à la coordination) en un vecteur latent global. Ce vecteur servira ensuite à piloter les blocs "FiLM" du UNet.

Trois architectures sont possibles.

Les réseaux de neurones convolutifs 1D (1D-CNN) sont parfaits pour l'EXAFS. Les convolutions sont capables de détecter les oscillations (fréquences), les pics de coordination et les motifs locaux, indépendamment de leur position exacte sur le vecteur k . En physique, la fréquence des oscillations dans l'espace k est directement liée à la distance interatomique R . Les filtres de convolution d'un CNN 1D agissent mathématiquement comme des détecteurs de motifs oscillants. Le réseau apprendra à associer certaines fréquences de convolution à des distances spécifiques, simulant ainsi une sorte de "Transformée de Fourier interne".

Les alliages à haute entropie génèrent des spectres EXAFS très complexes avec un fort désordre structural entraînant des pics de transformée de Fourier souvent amortis ou superposés. Un `mini-ResNet` 1D avec des connexions résiduelles permet d'aller chercher des caractéristiques très subtiles sans souffrir du problème de disparition du gradient.

Un perceptron multicouche (MLP) classique traite le spectre comme un simple vecteur géant de caractéristiques indépendantes. L'inconvénient est qu'il perd la notion d'ordre et de continuité des points le long de l'axe k . En revanche si on l'entraîne non pas sur l'espace k (le signal $\chi(k)$), mais directement sur le spectre dans l'espace R (la Transformée de Fourier du spectre), le MLP est très efficace car les positions des pics (première couche de coordination, deuxième couche) sont fixes sur l'axe des R .

Selon que l'on travaille dans l'espace k ou l'espace R , l'information pertinente à extraire des signaux EXAFS diffère. Dans l'espace k , le signal utile réside entièrement dans la fréquence, l'amplitude et le déphasage des oscillations du signal $\chi(k)$ (ou plus précisément de sa forme pondérée $k^2\chi(k)$ ou $k^3\chi(k)$). Pour l'espace k , le CNN 1D est le choix le plus puissant et le plus adapté, pour deux raisons physiques et mathématiques :

- **Détection des fréquences (Distances) :** En physique, la fréquence des oscillations dans l'espace k est directement liée à la distance interatomique R . Les filtres de convolution d'un CNN 1D agissent mathématiquement comme des détecteurs de motifs oscillants. Le réseau apprendra à associer certaines fréquences de convolution à des distances spécifiques, simulant ainsi une sorte de "Transformée de Fourier interne".
- **Robustesse au déphasage :** Le `Global Average Pooling` à la fin du CNN permet de capturer la présence d'une fréquence ou d'une signature chimique, peu importe où l'oscillation commence ou s'atténue sur l'axe k .

En entrée, on injecte une matrice de spectres (contenant chacun N_{points}) de taille $N_{\text{points}} \times N_{\text{espèces}}$. Le rôle de l'encodeur est de compresser ce signal hautement dimensionnel en un vecteur "latent" compact (ex : 128 ou 256 valeurs) qui représente l' "empreinte digitale" chimique et structurale locale de la particule.

A priori, il est conseillé d'injecter dans le réseau le signal déjà multiplié par k^2 ou k^3 ([A vérifier](#)). Les oscillations à haut k contiennent l'information sur la nature des atomes rétrodiffuseurs[1]. Si on injecte le signal non pondéré, le réseau va ignorer la fin du spectre parce que son amplitude est trop faible par rapport au début.

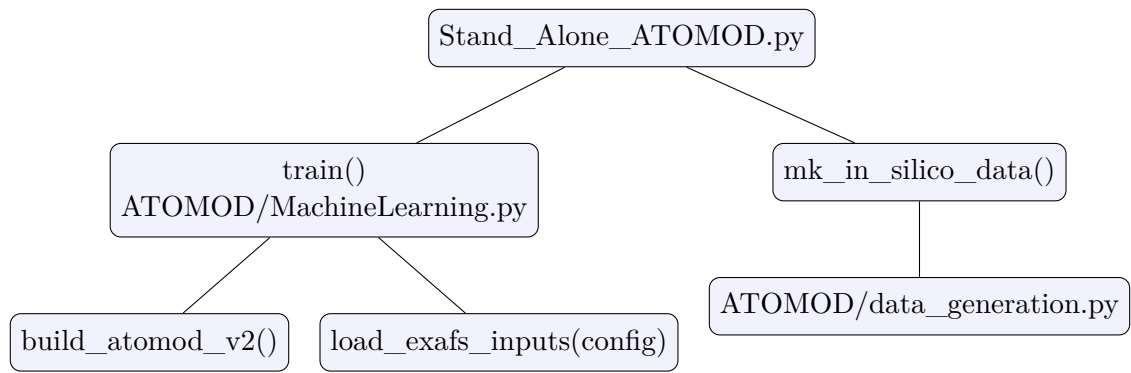


FIG. 3.1 : Structure du code `Stand_Alone_ATOMOD.py`

3.6 Le UNet de reconstruction

C'est un UNet standard, mais ses blocs de convolution sont modifiés pour accepter le vecteur latent EXAFS. Nous utilisons la technique **FiLM** ("Feature-wise Linear Modulation"). À chaque étage de l'encodeur et du décodeur du UNet, tu utilises le vecteur latent EXAFS pour générer des paramètres γ et β qui vont normaliser et décaler les cartes de caractéristiques ("feature maps") de l'image.

Chapitre 4

Utilisation du code FEFF dans le projet M2P2_HEA

4.1 Objectif scientifique

Dans le cadre du projet M2P2_HEA, le code de calcul *ab initio* FEFF est utilisé pour générer une base de données massive et standardisée de spectres d'absorption de rayons X (XAS/EXAFS). Cette base de données constitue le socle d'apprentissage du réseau de neurones ATOMOD, dont l'objectif est la reconstruction structurale 3D et l'identification chimique des nanoparticules d'alliages à haute entropie (HEA).

4.2 Méthodologie de Simulation

Le flux de travail repose sur une approche de modélisation "bottom-up". Des modèles atomiques de nanoparticules multimétalliques sont générés avec des configurations de taille (N), de forme et de degré d'alliage variables (coordonnées x, y, z) (Fig. 4.2(a)). Puis, pour chaque atome i (espèce chimique X) composant la nanoparticule, FEFF calcule la fonction de modulation $\chi_i^{(X)}(k)$ résultant de l'interférence du photo-électron éjecté de l'atome i avec son environnement local+ (Fig. 4.2(b,c,d)). Afin de reproduire le signal expérimental (qui est une mesure macroscopique), le script automatise le calcul de chaque atome de la nanoparticule en tant qu'absorbeur potentiel. Le spectre final est obtenu par une moyenne pondérée :

$$\chi_{\text{global}}^{(X)}(k) = \frac{1}{N_X} \sum_{i=1}^{N_X} \chi_i^{(X)}(k) \quad (4.1)$$

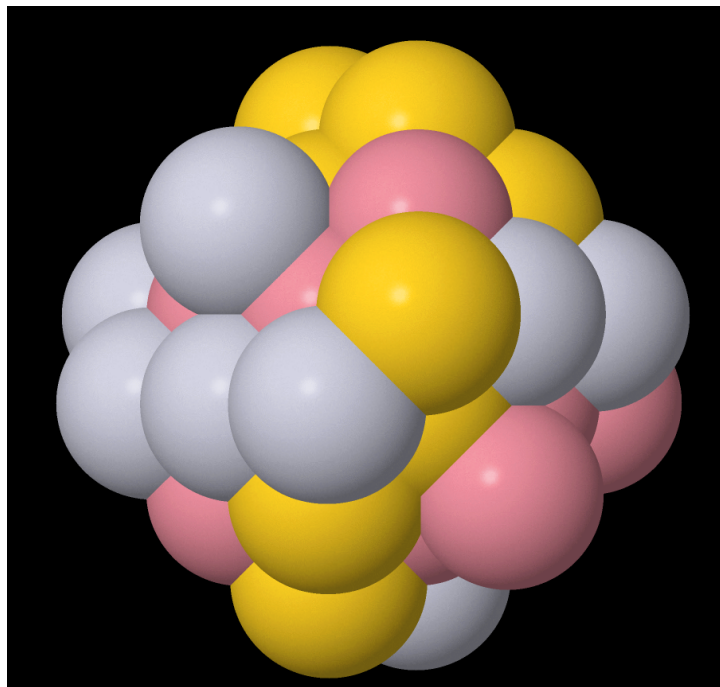
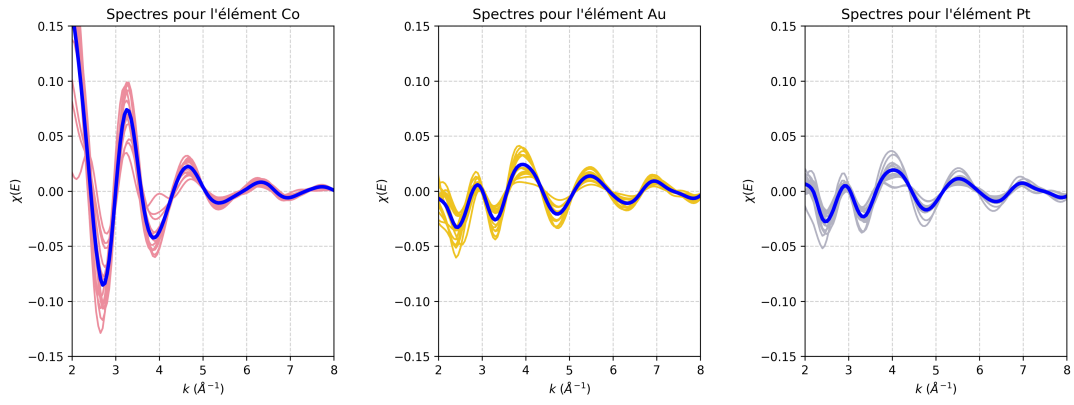
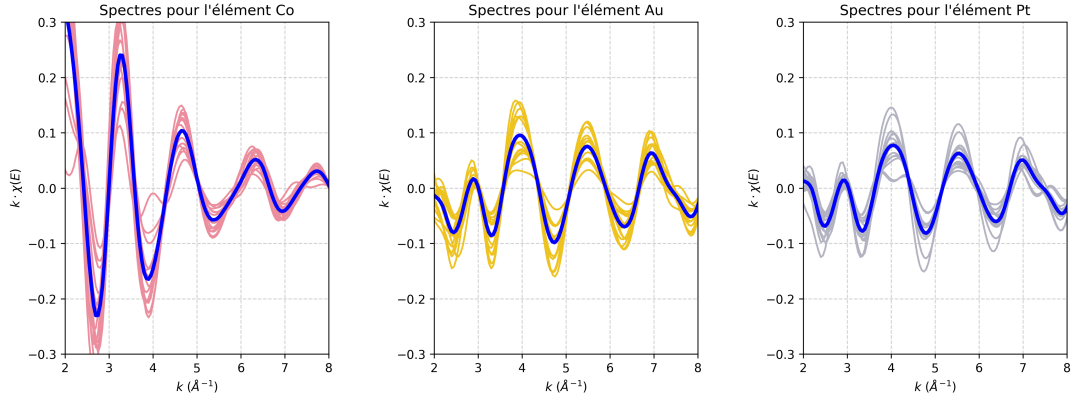


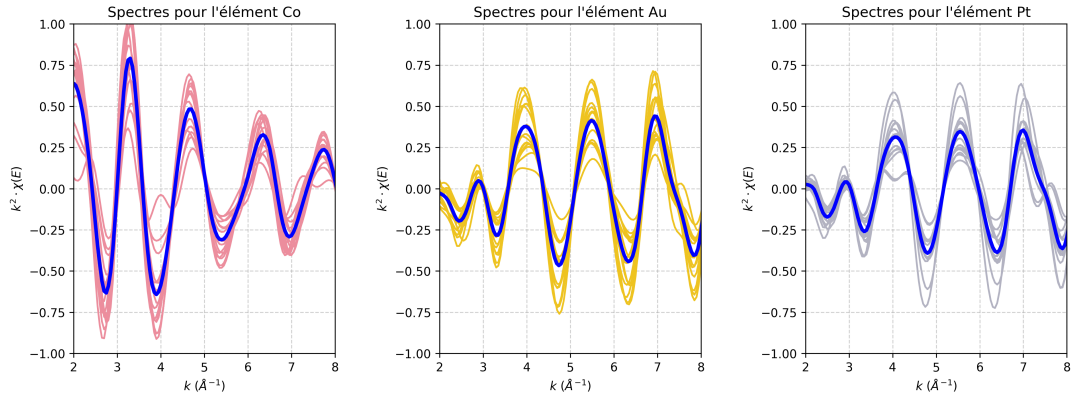
FIG. 4.1 : Vue 3D d'une particule $AuCoPt$.



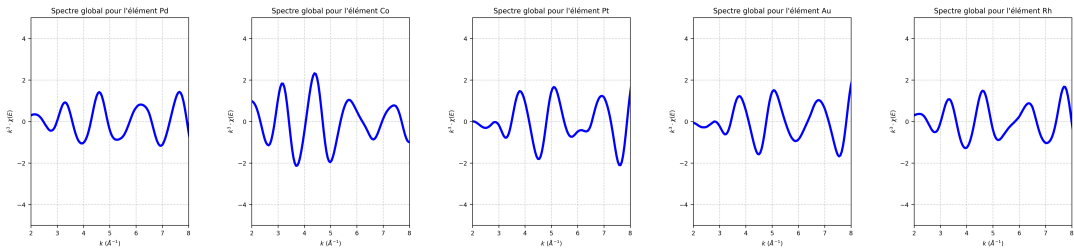
(a) Spectres $\chi(k)$.



(b) Spectres $k \cdot \chi(k)$.



(c) Spectres $k^2 \cdot \chi(k)$.



(d) Spectres $k^3 \cdot \chi(k)$.

FIG. 4.2 : Pour chaque courbe représentant le signal EXAFS, la courbe en bleu représente le signal global émis par l'ensemble de la nanoparticule. Les autres couleurs correspondant chacune au signal émis par un atome appartenant à la nanoparticule.

Le script `jupyter tuto-mk_mean.ipynb` illustre le "workflow" pour générer les signaux EXAFS émis par une nanoparticule d'alliage *AuPtCo*.

4.3 Structure du fichier de données générées par FEFF

En sortie d'un calcul EXAFS (module final `ff2x`), FEFF génère le fichier `xmu.dat`. Ce fichier est structuré sous forme de tableau en colonnes textuelles, précédé d'un en-tête (lignes commençant par `#`) qui récapitule les paramètres de la simulation (seuil, potentiels, etc.).

Numéro de colonne	Index Python (col)	Nom mathématique FEFF	Description et Unité
1	0	<code>omega</code> (ω)	Énergie absolue du photon incident (en eV). C'est l'axe des X idéal pour tracer le spectre d'absorption complet (XANES).
2	1	<code>e</code> ($E - E_0$)	Énergie relative par rapport au seuil (en eV). Vaut 0 exactement au seuil d'absorption (E_0).
3	2	<code>k</code> (k)	Vecteur d'onde du photo-électron (en $\text{\AA}^{-1}$). C'est l'axe des X indispensable pour tracer le signal d'oscillations $\chi(k)$.
4	3	<code>mu</code> (μ)	Coefficient d'absorption total calculé (avec les oscillations).
5	4	<code>mu0</code> (μ_0)	Coefficient d'absorption atomique de fond (le spectre "lissé", sans les voisins). C'est l'équivalent de la ligne de base ("spline").
6	5	<code>chi</code> (χ)	Le signal EXAFS pur (les oscillations isolées et normalisées). Mathématiquement, il correspond à : $\chi = \frac{\mu - \mu_0}{\mu_0}$.

Annexe A

Glossaire

Architecture On parle d'architecture pour désigner la structure théorique du réseau. C'est un concept abstrait, indépendant du problème ou des données. Par exemple, "l'architecture UNet" fait référence à la forme caractéristique en "U" du UNet, avec sa branche descendante (encodeur), sa branche ascendante (décodeur) et ses connexions sautées (skip connections).

[>] To Do

Insérer une figure du UNet

BATCH_SIZE Le BATCH_SIZE (Taille du Lot) est l'un des hyperparamètres les plus fondamentaux et les plus importants dans l'entraînement des réseaux de neurones. Il correspond au nombre d'échantillons de données (ici, d'images et de leurs 10 masques cibles) que le modèle va traiter simultanément avant d'ajuster ses poids.

EPOCHS Une époque représente un cycle complet d'entraînement. Une époque est définie comme un passage complet de l'ensemble du jeu de données d'entraînement à travers le réseau de neurones.

Modèle On parle de modèle (ou de modèle entraîné) dès que l'architecture est concrétisée : elle a des dimensions spécifiques (ex : des images de 128×128 en entrée, 15 canaux en sortie) et, surtout, elle possède des poids numériques appris après l'entraînement. C'est l'objet mathématique final sauvegardé dans un fichier `.h5` ou `.keras` et que l'on utilise pour faire des prédictions à partir des images TEM et des spectres EXAFS.

Le spectre génère des coefficients qui multiplient et s'ajoutent aux cartes de caractéristiques de l'image ($y = \gamma \cdot x + \beta$).

Bibliographie

- [1] Matthew Newville. Fundamentals of XAFS. *Consortium for Advanced Radiation Sources, University of Chicago (USA)*[<http://xafs.org>], 78, 2004.